Proyecto 2 — Entrega E1

Aedo Farge, Gian Franco
Colque Ordoñez, Patrick Fabrizio
Sander's Arana, Ralfh Krishna Karl

Arquitectura de Computadoras, noviembre de 2026

Resumen

Implementamos un pipeline RV32I de cinco etapas con forwarding, stalling y flushing, y lo verificamos con cuatro programas de prueba que pasan sus testbenches automáticos. En el camino encontramos y corregimos un error por el que `jalr` no anulaba las instrucciones del camino incorrecto. Sobre ese pipeline empezamos la arquitectura Dual-Issue: el procesador ya lee y decodifica dos instrucciones por ciclo y tiene un Register File de cuatro lecturas y dos escrituras. También quedaron instanciados el registro ID/EX y la ALU de la segunda vía. Por ahora el procesador sigue emitiendo una instrucción por ciclo, en orden.

1. Organización del trabajo

El proyecto termina comparando tres arquitecturas: (A) el pipeline Single-Issue, (B) el Dual-Issue y (C) el Dual-Issue optimizado. Para que esa comparación sea justa, cada arquitectura se conserva como una versión separada del código y ninguna modifica a la anterior. En esta entrega hay dos: la versión Single-Issue, que es la arquitectura A ya terminada, y la versión Dual-Issue, que parte de ella y contiene el avance hacia B. Todas las waveforms de este informe se obtuvieron simulando los testbenches incluidos en el código, con un reloj de 10 ns; el reset se desactiva a los 22 ns.

2. El pipeline de cinco etapas

Cada instrucción pasa por cinco etapas separadas por registros de pipeline (Tabla 1). Como en cada ciclo hay hasta cinco instrucciones avanzando a la vez, en el caso ideal se termina una instrucción por ciclo (CPI = 1). El procesador soporta `add`, `sub`, `and`, `or`, `xor`, `slt`, sus versiones con inmediato, `lw`, `sw`, `beq`, `jal` y `jalr`.

Tabla 1: Etapas del pipeline.

Etapa	Qué hace	Registro
IF	Lee la instrucción en `PCF` y calcula `PC+4`.	IF/ID
ID	Decodifica, lee `rs1` y `rs2` del Register File y extiende el inmediato.	ID/EX
EX	Opera la ALU, calcula el destino del salto y decide si se toma (`PCSrcE`).	EX/MEM
MEM	`lw` lee y `sw` escribe en la memoria de datos.	MEM/WB
WB	Elige el resultado (ALU, dato leído o `PC+4`) y lo escribe en el Register File.	—

2.1 Datapath

La Figura 1 muestra el datapath. Las señales llevan como sufijo la etapa donde están (`F`, `D`, `E`, `M`, `W`). Además de datos y control, los registros de pipeline propagan la instrucción completa (`InstrD` a `InstrW`); no la necesita el hardware, pero permite ver en las waveforms en qué etapa está cada instrucción.

El multiplexor del PC tiene tres entradas: `00` para `PC+4`, `01` para el destino de `beq` y `jal` (`PCTargetE`) y `10` para `jalr`, cuyo destino es `ALUResultE` con el bit 0 en cero. El Register File escribe en el flanco de bajada del reloj: así, una instrucción en WB escribe en la primera mitad del ciclo y la que está en ID lee el valor

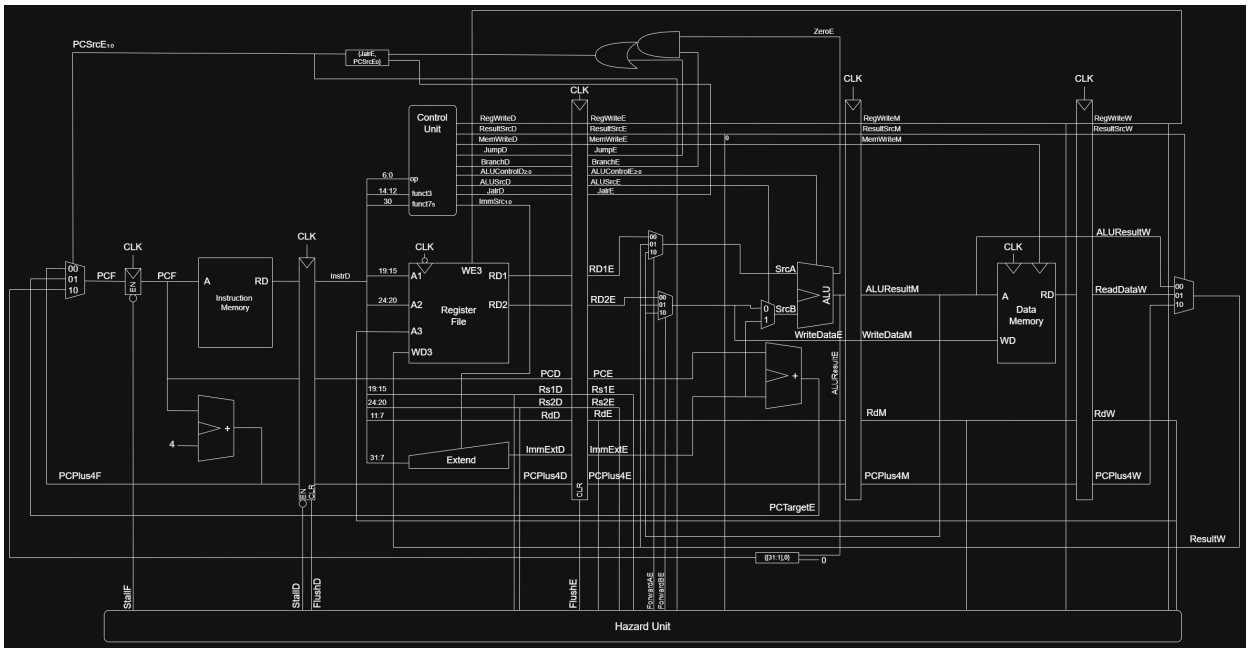


Figura 1: Datapath del pipeline Single-Issue con la Hazard Unit: registros de pipeline, multiplexores de forwarding en EX y señales de stall y flush.

nuevo en la segunda mitad, lo que nos ahorra un caso de forwarding. Cuando el registro IF/ID se anula, se carga un NOP real (`addi x0, x0, 0`) en lugar de ceros, para que el controller nunca decodifique una instrucción inválida.

2.2 Unidad de control

El controller decodifica en ID con `maindec` y `aludec` y hace viajar cada señal con su instrucción hasta la etapa donde se usa. `ImmSrc` se consume en ID; `ALUSrc`, `ALUControl`, `Branch`, `Jump` y `Jalr` en EX; `MemWrite` en MEM, y `RegWrite` y `ResultSrc` en WB. El salto se resuelve en EX con `PCSrcE = {JalrE, (BranchE & ZeroE) | JumpE}`. El registro de control ID/EX se limpia con `FlushE`, de modo que una instrucción anulada no escribe registros ni memoria. Respecto al procesador single-cycle original, añadimos la señal `Jalr` a `maindec` (la palabra de control pasó de 11 a 12 bits) para soportar `jalr`.

3. Hazard Unit

La Hazard Unit (`hazardunit.v`) se encarga de los tres tipos de riesgo del pipeline.

Forwarding. Si un registro fuente de la instrucción en EX es el destino de una instrucción en MEM o WB que escribe en el Register File, se le pasa el valor directamente. MEM tiene prioridad porque contiene el resultado más reciente:

```
if      ((Rs1E != 0) && (Rs1E == RdM) && RegWriteM) ForwardAE = 2'b10; // desde MEM
else if ((Rs1E != 0) && (Rs1E == RdW) && RegWriteW) ForwardAE = 2'b01; // desde WB
else                                                ForwardAE = 2'b00;
```

Stall (load-use). Un `lw` recién tiene su dato al final de MEM. Si la instrucción que le sigue lo necesita, se congelan el PC y el registro IF/ID durante un ciclo y se ingresa una burbuja en EX:

```
assign lwStall = ResultSrcE0 & (RdE != 5'b0) & ((Rs1D == RdE) | (Rs2D == RdE));
assign StallF  = lwStall;
assign StallD  = lwStall;
```

Flush. Para instrucciones del formato B o de salto, el salto se decide en EX, pero para entonces ya entraron dos instrucciones (en IF e ID). Si el salto se toma, ambas se anulan, realizando el Flush con los registros IF/ID e ID/EX:

```
assign FlushD = |PCSrcE;
assign FlushE = lwStall | (|PCSrcE);
```

Error corregido: `jalr` no hacía flush

La versión original tenía `assign FlushD = PCSrcE;`. `PCSrcE` es de 2 bits y `FlushD` de 1, así que Verilog se quedaba solo con el bit menos significativo. Para `jalr` (`PCSrcE = 10`) eso daba `FlushD = 0`, y las dos instrucciones siguientes se ejecutaban aunque no debían. Lo corregimos con la reducción OR (`|PCSrcE`), que vale 1 ante cualquier salto tomado. El Programa 4 lo comprueba: con la versión anterior, `x14` y `x15` terminaban en 99; con la corregida quedan en 0. También agregamos `RdE != 0` a `lwStall`, porque un `lw` hacia `x0` no debería frenar el pipeline.

4. Programas de prueba

Cada programa tiene su código Assembly, su archivo `.mem` (verificado contra el ensamblador RISC-V) y un testbench que revisa por sí solo los registros, la memoria y la cantidad de stalls y flushes. La Tabla 2 resume lo obtenido; en las secciones siguientes explicamos cada waveform.

Tabla 2: Resultados de los programas de prueba sobre el pipeline Single-Issue.

Programa	Qué prueba	Qué se observa	Testbench
1 — ISA	Todas las instrucciones, sin dependencias	Sin forwarding, stall ni flush	PASS
2 — Forwarding	Cadena de dependencias RAW	4 ciclos con forwarding	PASS
3 — Stalling	Dependencia load-use	1 stall	PASS
4 — Flushing	`beq`, `jal`, `jalr`	3 flushes	PASS

4.1 Programa 1: instrucciones sin dependencias

Este programa recorre todas las instrucciones que soporta el procesador y las ordena para que ninguna necesite a la Hazard Unit: cada instrucción que produce un valor está al menos a tres instrucciones de la que lo usa, distancia a la que el Register File ya tiene el dato gracias a la escritura en flanco de bajada. `jal`, `jalr` y un `beq` tomado no pueden probarse aquí, porque siempre provocan un flush; esos se verifican en el Programa 4. El testbench comprueba los registros `x1` a `x18`, las dos palabras escritas en memoria y que durante todo el programa no haya ningún forwarding, stall ni flush.

```
addi x4, x0, 3       # x4 = 3
addi x1, x0, 12      # x1 = 12
addi x2, x0, 5       # x2 = 5
addi x3, x0, 8       # x3 = 8
                     # 4 NOP
add  x5, x1, x2      # x5 = 17
sub  x6, x3, x4      # x6 = 5
and  x7, x1, x3      # x7 = 8
or   x8, x2, x4      # x8 = 7
xor  x9, x1, x2      # x9 = 9
slt  x10, x4, x2     # x10 = 1   (3 < 5)
addi x11, x0, -5     # x11 = -5
andi x12, x1, 6      # x12 = 4   (12 & 6)
ori  x13, x2, 8      # x13 = 13  (5 | 8)
xori x14, x3, 15     # x14 = 7   (8 ^ 15)
slti x15, x2, 7      # x15 = 1   (5 < 7)
sw   x1, 0(x0)       # Mem[0] = 12
sw   x2, 4(x0)       # Mem[1] = 5
lw   x16, 0(x0)      # x16 = 12
lw   x17, 4(x0)      # x17 = 5
                     # 3 NOP
beq  x16, x17, salto  # 12 != 5: no se toma
addi x18, x0, 1      # x18 = 1
salto:
```



Figura 2: Programa 1. Arriba, de 20 a 200 ns; abajo, de 195 a 345 ns.

En la Figura 2 se ve el caso ideal del pipeline: cada instrucción baja en diagonal de `InstrF` a `InstrW`, una etapa por ciclo, y el PC avanza de 4 en 4 sin detenerse. `StallF`, `FlushD`, `ForwardAE`, `ForwardBE` y `PCSrcE` se mantienen en 0 durante todo el programa, así que la Hazard Unit nunca interviene. `ResultW` muestra los valores en el orden en que se escriben: 3, 12, 5 y 8 de la inicialización (60–90 ns); 17, 5, 8, 7, 9 y 1 de las instrucciones tipo R, incluido el `slt` (140–190 ns), y −5, 4, 13, 7 y 1 de las instrucciones con inmediato (200–240 ns).

La parte de memoria está en la captura inferior. `MemWrite` sube durante dos ciclos, entre los 235 y 255 ns: son los dos `sw`, que escriben 12 en `Mem[0]` y 5 en `Mem[1]`. A los 270 y 280 ns, `ResultW` muestra 12 y 5, los valores que los `lw` leyeron de esas mismas direcciones. El `beq` llega a EX a los 300 ns; como 12 ≠ 5,

`PCSrcE` sigue en `00`, no hay flush, y `addi x18` escribe su 1 a los 330 ns. Los valores 0 y 4 de `ResultW` hacia los 250 ns son las direcciones que calcularon los `sw`, y el 7 de los 320 ns es la resta 12 – 5 del `beq`; ninguno se escribe, porque esas instrucciones tienen `RegWrite = 0`.

4.2 Programa 2: forwarding

```
addi x1, x0, 5      # x1 = 5
addi x2, x0, 10     # x2 = 10
add  x3, x1, x2     # x3 = 15   (x1 desde WB, x2 desde MEM)
sub  x4, x3, x1     # x4 = 10   (x3 desde MEM)
add  x5, x4, x3     # x5 = 25   (x4 desde MEM, x3 desde WB)
add  x6, x5, x4     # x6 = 35   (x5 desde MEM, x4 desde WB)
```

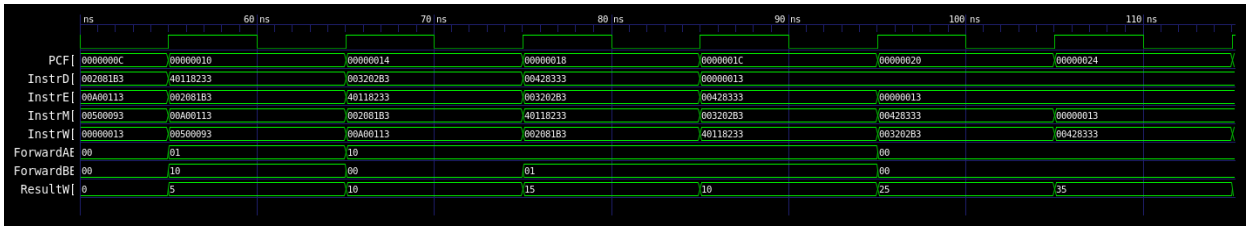


Figura 3: Programa 2, de 50 a 115 ns.

Aquí cada instrucción necesita el resultado de la anterior antes de que llegue al Register File. La Tabla 3 resume lo que hace el forwarding en cada ciclo de la Figura 3. Ninguna instrucción se detiene: sin forwarding, cada una habría esperado dos ciclos. `ResultW` confirma los valores 15, 10, 25 y 35 entre los 80 y 110 ns.

Tabla 3: Forwarding observado en el Programa 2.

Tiempo	En EX	ForwardAE	ForwardBE	De dónde vienen los operandos
60 ns	`add x3, x1, x2`	01	10	`x1` desde WB, `x2` desde MEM
70 ns	`sub x4, x3, x1`	10	00	`x3` desde MEM
80 ns	`add x5, x4, x3`	10	01	`x4` desde MEM, `x3` desde WB
90 ns	`add x6, x5, x4`	10	01	`x5` desde MEM, `x4` desde WB

4.3 Programa 3: stalling por load-use

```
addi x1, x0, 0      # x1 = 0
addi x2, x0, 10     # x2 = 10
sw   x2, 0(x1)      # Mem[0] = 10
lw   x3, 0(x1)      # x3 = 10
add  x4, x3, x2     # x4 = 20   <- usa x3 justo despues del lw: stall
sub  x5, x4, x2     # x5 = 10
add  x6, x5, x3     # x6 = 20
```

La Figura 4 muestra el único stall del programa. A los 60 ns, el `sw` recibe por forwarding desde MEM el dato que va a guardar (`ForwardBE = 10`). A los 70 ns el `lw x3` está en EX y el `add x4, x3, x2` en ID; como `x3` recién estará listo al final de MEM, la Hazard Unit levanta a la vez `StallF`, `StallD` y `FlushE`. El efecto se ve claro a los 80 ns: `PCF` sigue en `0x14` e `InstrD` sigue siendo el `add`, porque ambos quedaron congelados, mientras `InstrE` vale 0, que es la burbuja insertada.

Un ciclo después, a los 90 ns, el `add` entra a EX y toma `x3 = 10` desde WB (`ForwardAE = 01`), porque el `lw` ya leyó la memoria. Las dos instrucciones siguientes obtienen su operando desde MEM (`ForwardAE = 10`) sin más esperas. Un ciclo de stall es el costo mínimo de una dependencia load-use en este pipeline.

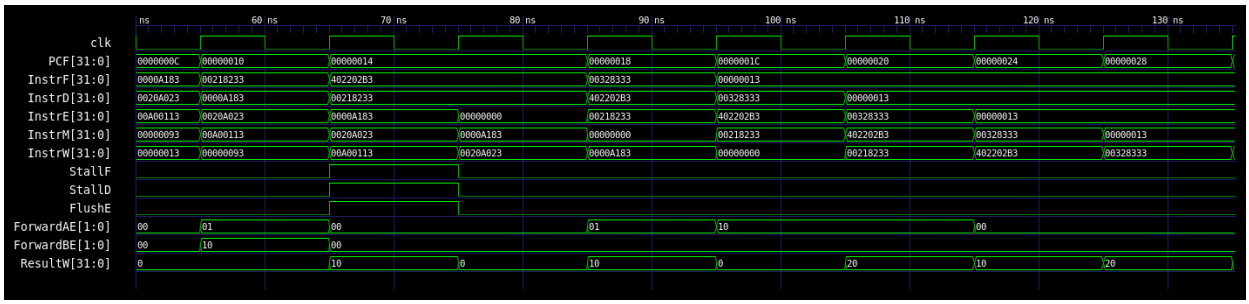


Figura 4: Programa 3, de 50 a 135 ns.

4.4 Programa 4: flushing

```
      addi x1, x0, 5
      addi x2, x0, 5
      addi x3, x0, 7
      beq  x1, x3, t1      # NO se toma (5 != 7)
      addi x4, x0, 1       # x4 = 1
      beq  x1, x2, t1      # se toma -> flush
      addi x5, x0, 99      # anulada
      addi x6, x0, 99      # anulada
t1:   addi x7, x0, 2
      jal  x8, t2          # x8 = 40 -> flush
      addi x9, x0, 99      # anulada
      addi x10, x0, 99     # anulada
t2:   addi x11, x0, 3
      addi x12, x0, 68
      jalr x13, 0(x12)     # x13 = 60 -> flush
      addi x14, x0, 99     # anulada
      addi x15, x0, 99     # anulada
t3:   addi x16, x0, 4
      sw   x16, 0(x0)
fin:  beq  x0, x0, fin
```

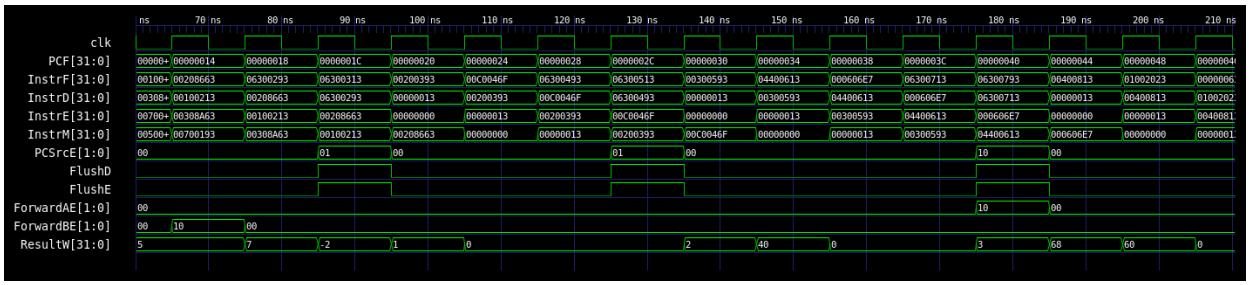


Figura 5: Programa 4, de 60 a 210 ns.

El programa recorre los tres tipos de salto. En la Figura 5, a los 70 ns el primer `beq` está en EX y recibe `x3` por forwarding (`ForwardBE = 10`); como 5 ≠ 7, `PCSrcE` queda en `00` y no pasa nada: `addi x4` sigue su camino.

A los 90 ns el segundo `beq` sí se toma. `PCSrcE` pasa a `01` y se activan `FlushD` y `FlushE`, que anulan las dos `addi ..., 99` que estaban en ID e IF. En el ciclo siguiente se ve el resultado: `InstrD` es un NOP, `InstrE` vale 0 y `PCF` ya apunta a `0x20` (`t1`). El `jal` de los 130 ns se comporta igual, y a los 150 ns `ResultW` muestra el 40 que se guarda en `x8` como dirección de retorno.

El caso más interesante es el `jalr` de los 180 ns. Recibe `x12 = 68` por forwarding desde MEM (`ForwardAE = 10`), pone `PCSrcE = 10` y esta vez sí provoca el flush, que es justamente lo que fallaba antes de la corrección. A los 190 ns el PC ya está en `0x44` (`t3`) y a los 200 ns `ResultW` muestra el 60 de `x13`.

El −2 que aparece en `ResultW` hacia los 90 ns no es un error: es la resta 5 − 7 que hizo el `beq` no tomado al llegar a WB, y no se escribe en ningún registro porque `beq` tiene `RegWrite = 0`. Cada salto tomado cuesta dos ciclos, uno por cada instrucción anulada.

5. Primeros pasos hacia Dual-Issue

En esta entrega construimos la parte frontal del procesador Dual-Issue: leer y decodificar dos instrucciones por ciclo. La ejecución en paralelo (Issue Unit y segunda ALU) queda para E2. La Figura 6 resume los cambios.

Dual Fetch. A la memoria de instrucciones le agregamos un segundo puerto de lectura: `rd` entrega la instrucción en `PC` y `rd1` la de `PC+4`. Como en cada lectura llega un par, el PC avanza de 8 en 8. El PC se actualiza cuando el par que está en Decode ya terminó de emitirse o cuando hay un salto tomado.

Dual Decode. El registro IF/ID ahora es doble y guarda `InstrD0` (slot 0, la instrucción más antigua), `InstrD1` (slot 1) y el PC base del par. El controller tiene dos decodificadores (dos `maindec` y dos `aludec`) y el datapath dos unidades `extend`, así que ambas instrucciones se decodifican en el mismo ciclo. El Register File pasó a tener cuatro puertos de lectura y dos de escritura: lee `rs1` y `rs2` de las dos instrucciones a la vez. El segundo puerto de escritura ya existe y, si las dos vías escriben el mismo registro, gana la vía 1 por ser la instrucción más joven; lo conectaremos en E2.

Módulos de la vía 1. También instanciamos el registro ID/EX de la vía 1 (`regE1`, con los operandos, el inmediato y el control del slot 1) y su ALU (`alu1`), con su propio multiplexor de `ALUSrc`. En esta entrega la señal `Issue1D` está fija en 0, de modo que ese registro se limpia en cada ciclo y la vía 1 solo procesa burbujas. En E2 la Issue Unit pondrá `Issue1D` en 1 cuando las dos instrucciones del par puedan salir juntas.

Emisión. Como E1 todavía no pide ejecutar dos instrucciones a la vez, emitimos una por ciclo y en orden. La señal `SelD` elige el slot: con `SelD = 0` sale la instrucción más antigua del par y con `SelD = 1` la más joven; después entra el siguiente par. A EX pasan el control y los operandos del slot elegido. Esta idea de guardar la instrucción más joven para el ciclo siguiente es la base del reagrupamiento que hará la Issue Unit en E2. Ante un salto tomado se limpia el par en Decode, y si el salto estaba en el slot 0, la instrucción del slot 1 se anula con `FlushE`.

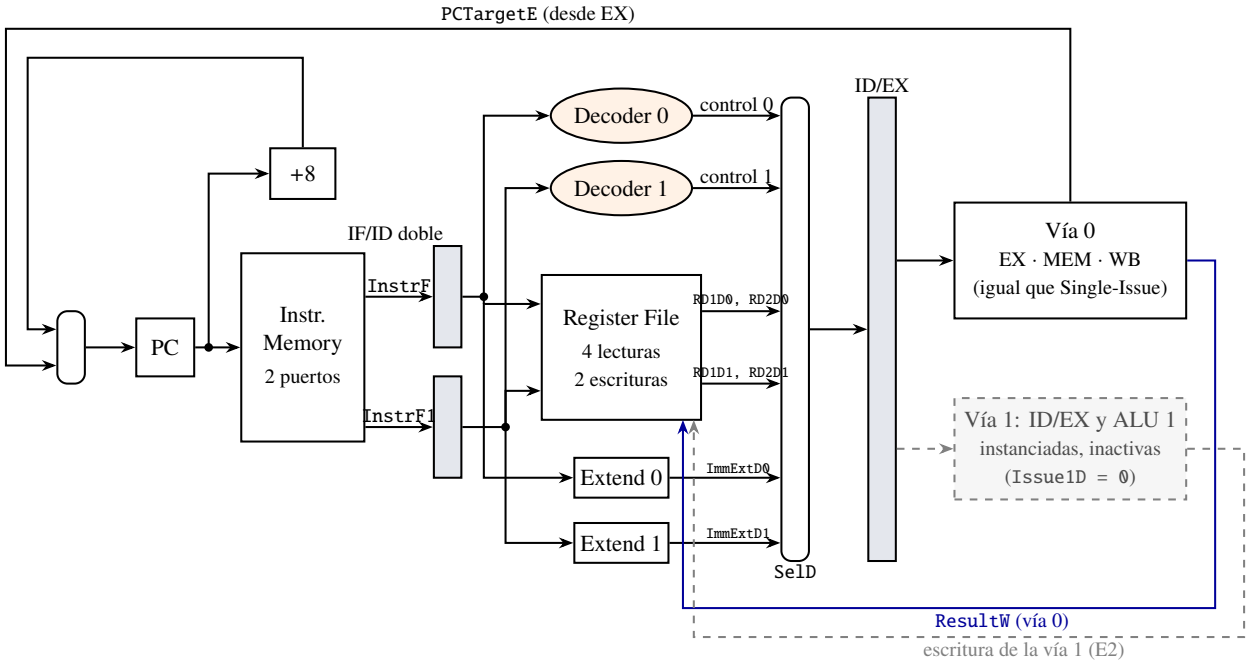


Figura 6: Datapath Dual-Issue preliminar. El par se lee y decodifica en paralelo y el multiplexor controlado por `SelD` elige qué instrucción pasa a EX. En gris, la vía 1: su registro ID/EX y su ALU ya están instanciados, pero en E1 solo reciben burbujas.

5.1 Programa 5: evidencia de Dual Fetch y Dual Decode

```
        addi x1, x0, 1      # par 0x00
        addi x2, x0, 2
        addi x3, x0, 3      # par 0x08
        addi x4, x0, 4
        add  x5, x1, x2     # par 0x10
        add  x6, x3, x4
        sw   x5, 0(x0)      # par 0x18
        sw   x6, 4(x0)
        beq  x0, x0, t1     # par 0x20: salto tomado en el slot 0
        addi x7, x0, 99     #           el slot 1 se anula
t1:     addi x8, x0, 8
fin:    beq  x0, x0, fin
```

El testbench de este programa revisa cinco cosas: que el par leído sea siempre (PC, PC+4); que sin saltos el PC avance de 8 en 8; que después de cada salto el PC tome el destino correcto; que las 11 instrucciones válidas lleguen a EX en orden, sin perderse ni repetirse, y que registros y memoria terminen con los valores esperados. Los cuatro programas anteriores también pasan sin cambios sobre esta versión, así que el Dual Fetch y el Dual Decode no alteraron el comportamiento del procesador.

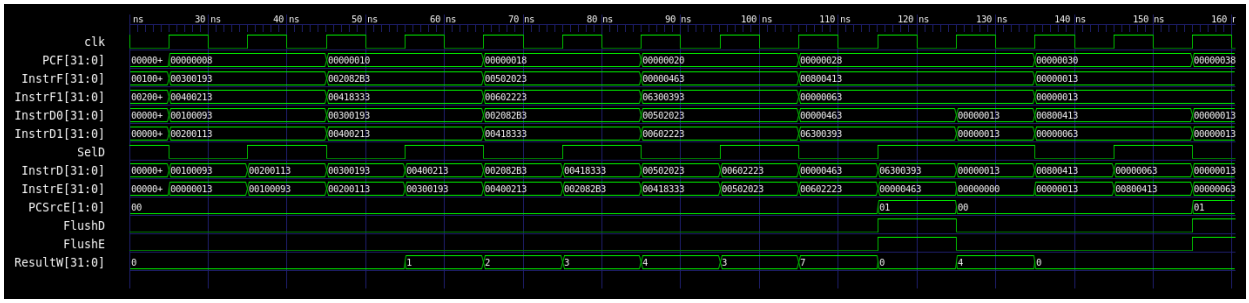


Figura 7: Programa 5, de 20 a 160 ns.

En la Figura 7 se ve el ritmo del Dual Fetch. A los 30 ns, Decode tiene el par `0x00` (`InstrD0 = addi x1`, `InstrD1 = addi x2`) y, al mismo tiempo, Fetch ya está leyendo el par `0x08` (`InstrF = addi x3`, `InstrF1 = addi x4`). Con `SelD = 0` sale `addi x1`; a los 40 ns, con `SelD = 1`, sale `addi x2`, mientras el PC y el par de Fetch esperan. A los 50 ns entra el par `0x08` a Decode y `PCF` salta de `0x08` a `0x10`. El patrón se repite cada dos ciclos: un par leído y decodificado, dos instrucciones emitidas.

El salto aparece a los 110 ns, cuando entra el par `0x20` con el `beq` en el slot 0 y `addi x7, 99` en el slot 1. El `beq` se emite primero y a los 120 ns se resuelve tomado en EX (`PCSrcE = 01`) justo cuando se está emitiendo `addi x7`. `FlushE` la anula (a los 130 ns `InstrE` vale 0) y `FlushD` limpia el par. A los 140 ns entra el par destino (`addi x8` y `beq fin`) y todo sigue en orden.

Un error que encontramos con este programa

Al revisar la simulación vimos que, si un salto se tomaba mientras se emitía el slot 0, el PC no saltaba: solo se actualizaba cuando `SelD = 1`, y el destino se perdía. Lo corregimos permitiendo que el PC se actualice también cuando hay un salto tomado (`en = ~StallF & (SelD | FlushD)`), y agregamos al testbench la verificación del destino después de cada salto, que con la versión anterior falla.

6. Conclusiones

El pipeline de cinco etapas ejecuta correctamente el subconjunto RV32I que soporta y resuelve los tres riesgos del pipeline: forwarding desde MEM y WB, un ciclo de stall ante load-use y la anulación de dos instrucciones en cada salto tomado. Las pruebas automáticas fueron clave: gracias a ellas encontramos dos

errores, el flush que faltaba en `jalr` y la pérdida del destino del salto en el Dual Fetch, que con una revisión visual de las waveforms habrían sido fáciles de pasar por alto.

La base del Dual-Issue ya está: Fetch y Decode trabajan con pares de instrucciones y el Register File tiene los cuatro puertos de lectura y dos de escritura que se van a necesitar. La vía 1 ya tiene su registro ID/EX y su ALU. Para E2 queda lo más interesante: completar esa vía hasta WB, la Issue Unit que decida si dos instrucciones pueden salir juntas (dependencias RAW dentro del par y recursos compartidos de memoria y branch) y el forwarding entre vías.

A. Cómo reproducir las pruebas en Vivado

Cada programa está en su propia carpeta, con su código Assembly, su archivo `.mem` y su testbench (Tabla 4). Los programas 1 a 4 se simulan con la versión Single-Issue del código y el Programa 5 con la versión Dual-Issue; los programas 1 a 4 también pasan sin cambios en la versión Dual-Issue.

Tabla 4: Archivos de cada prueba.

Programa	Versión	Carpeta	Testbench	Programa cargado
1 (ISA)	Single-Issue	`Programa1_ISA`	`tb_isa.v`	`isa.mem`
2 (Forwarding)	Single-Issue	`Programa2_Forwarding`	`tb_forwarding.v`	`forwarding.mem`
3 (Stalling)	Single-Issue	`Programa3_Stalling`	`tb_stalling.v`	`stalling.mem`
4 (Flushing)	Single-Issue	`Programa4_Flushing`	`tb_flushing.v`	`flushing.mem`
5 (Dual Fetch)	Dual-Issue	`Programa5_DualFetch`	`tb_dualfetch.v`	`dualfetch.mem`

Para simular, por ejemplo, el Programa 1:

1. Crear un proyecto RTL nuevo en Vivado.
2. Agregar como *design sources* todos los archivos `.v` de la carpeta principal del código (de `adder.v` a `top.v`).
3. Agregar como *simulation sources* el testbench del programa y su `.mem` asociado: `Programa1_ISA/tb_isa.v` e `Programa1_ISA/isa.mem`. Marcar el testbench como *Set as Top*.
4. Ejecutar *Run Simulation* → *Run Behavioral Simulation*.

El testbench se detiene solo e imprime `PASS` o `FAIL` en la consola Tcl. En la ventana de waveform aparecen directamente `PCF`, `InstrF` a `InstrW` y las señales de la Hazard Unit, porque el testbench las declara en su nivel superior. Para los demás programas se cambian el testbench y el `.mem` por los de la Tabla 4.